

LEARNING RESOURCE GUIDE

Generative AI for Beginners | Professional Participant Workbook

Section	Section 3: Large Language Models (LLMs)
Lecture	Lecture 3.5: Embeddings and Vector Representations

Learning Objectives

By the end of this lecture and its accompanying guide, you will be able to:

- Explain what an embedding is and how it represents words, sentences, or documents as dense numerical vectors.
- Describe how the geometry of the embedding space captures semantic relationships, enabling similarity calculations and analogy arithmetic.
- Explain the purpose of positional embeddings and how they allow Transformer models to encode word order.
- Distinguish between static word embeddings (Word2Vec, GloVe) and contextual embeddings produced by LLMs.
- Identify practical applications of embeddings in semantic search, recommendation, classification, and retrieval-augmented generation.

Introduction

Language models do not understand words the way humans do. When a model processes the word 'king', it does not access a mental dictionary entry. Instead, it looks up a dense numerical vector — a list of hundreds or thousands of floating-point numbers — that encodes everything the model has learned about how that word relates to every other word in its vocabulary. These vectors are called embeddings, and they are the mechanism by which language models turn the unstructured, messy world of human language into the precise numerical structures that neural networks can compute over.

The remarkable property of well-trained embeddings is that their geometry reflects meaning. Words with similar meanings cluster together in the vector space. Relationships between concepts become directions in the space: the vector from 'king' to 'queen' is approximately the same as the vector from 'man' to 'woman'. This is not programmed by a human — it emerges from training on the patterns of language. The discovery that meaning could emerge spontaneously from the geometry of high-dimensional vector spaces was a foundational insight for modern NLP.

Embeddings are no longer just an internal model component — they are a product in their own right. Embedding APIs allow developers to convert any text into a vector and then use those vectors for semantic search, document clustering, recommendation systems, and retrieval-augmented generation (RAG). Understanding embeddings unlocks an entire class of AI applications that go far beyond chatbots and text generation.

Core Concepts Explained

What is an Embedding?

Definition: An embedding is a dense, fixed-size vector of real numbers that represents a piece of text (a word, sentence, paragraph, or document) in a high-dimensional space. Unlike one-hot encodings (which assign one position per vocabulary item, producing sparse vectors), embeddings are dense — every dimension carries information — and are learned from data.

Business relevance: Embeddings enable computers to work with text using the same mathematical operations used for any numerical data. Once text is embedded, you can compute distances, averages, clusters, and nearest neighbours. This powers semantic search (find documents by meaning, not keyword), recommendation systems (find items similar to what a user liked), and anomaly detection (flag documents that are dissimilar to a baseline).

Practitioner relevance: For application developers, embedding APIs (such as OpenAI's text-embedding-ada-002 or open-source models like sentence-transformers) allow any text to be converted into a 1,536-dimensional (or similar) vector. These vectors can then be stored in a vector database (Pinecone, Weaviate, pgvector) and searched using approximate nearest-neighbour algorithms — the foundation of RAG architectures.

Real-world example: A legal technology company embeds its entire case law database — hundreds of thousands of court opinions — using a sentence embedding model. When a lawyer types a research query, the query is embedded into the same vector space and the nearest-neighbour search returns the most semantically similar cases within milliseconds, regardless of whether the exact query words appear in the cases.

Risks or limitations: Embeddings reflect the biases present in their training data. If the embedding model was trained on text where certain groups are consistently associated with certain roles or attributes, those associations are encoded in the geometry of the embedding space and will propagate into downstream applications. This is a documented source of bias in embedding-based systems.

Semantic Similarity and Vector Arithmetic

Definition: In a well-trained embedding space, semantic similarity is captured by geometric proximity. Two pieces of text with similar meanings produce vectors that are close together (low cosine distance). The famous word analogy result — king - man + woman $\approx$ queen — demonstrates that meaningful semantic relationships are encoded as consistent vector directions, enabling arithmetic operations on meaning.

Business relevance: Semantic similarity is the foundation of embedding-based applications. Cosine similarity (the dot product of two unit vectors) is the standard measure: a cosine similarity of 1.0 means identical, 0.0 means unrelated, and -1.0 means opposite. Values above 0.8 are typically considered highly similar. This metric enables ranking search results by semantic relevance rather than keyword overlap.

Practitioner relevance: When building semantic search or recommendation systems, the quality of the embedding model matters as much as the search algorithm. Embeddings from domain-specialised models (trained on scientific, legal, or financial text) typically outperform general-purpose embeddings for domain-specific queries. Selecting and benchmarking the embedding model is a key design decision.

Real-world example: A customer support system embeds all historical support tickets and their resolutions. When a new ticket arrives, its embedding is computed and the five most similar historical tickets are retrieved. The agent sees these as suggested solutions, reducing time-to-resolution. The similarity calculation happens in milliseconds even across millions of historical tickets using a vector database.

Risks or limitations: Cosine similarity works in the embedding space, not in the original text space. Two texts can be superficially very different in wording but semantically close (paraphrases), or superficially similar in

wording but semantically distant (negations, sarcasm). Embedding-based systems handle paraphrase well but can struggle with negation and implicit meaning.

Contextual vs Static Embeddings

Definition: Static embeddings (Word2Vec, GloVe) assign a single fixed vector to each word regardless of context — the word 'bank' has the same embedding whether it appears in a financial or a geographical context. Contextual embeddings, produced by models like BERT and GPT, generate different vectors for the same word depending on the surrounding context, capturing polysemy and nuanced meaning.

Business relevance: Contextual embeddings represent the current state of the art for most NLP applications because they capture how meaning changes with context. Sentence-level or passage-level embeddings from models like BERT or modern sentence-transformers outperform word-level averages on most semantic similarity and retrieval benchmarks.

Practitioner relevance: For most production applications, developers should use sentence-level contextual embeddings rather than word-level static embeddings. Static embeddings (Word2Vec, GloVe) are computationally cheaper but less accurate. The choice depends on latency requirements and the acceptable quality threshold for the application.

Real-world example: 'I went to the bank to deposit money' and 'The river bank was muddy after the flood' produce very different contextual embeddings for the word 'bank', correctly representing its different meanings. A static embedding would assign the same vector in both sentences, potentially causing a semantic search for financial 'bank' queries to retrieve river geography documents.

Risks or limitations: Contextual embeddings are significantly more computationally expensive to produce than static embeddings, as they require a full forward pass through a neural network for each input. For applications that need to embed millions of documents, this inference cost is a significant engineering and budget consideration.

Positional Embeddings

Definition: The Transformer's self-attention mechanism is permutation-invariant — it processes all tokens simultaneously and, without modification, would treat 'the dog bit the man' identically to 'the man bit the dog'. Positional embeddings are added to token embeddings to inject information about each token's position in the sequence, enabling the model to encode word order.

Business relevance: Positional embeddings are what allow Transformer-based models to understand sentence structure, grammar, and the sequential nature of language. Without them, the model would be a bag-of-words model — powerful at understanding which words are present, but blind to their order and therefore unable to understand syntax or meaning-by-position.

Practitioner relevance: Different positional embedding schemes have different effective sequence length limitations. Original sinusoidal positional embeddings (used in the original Transformer) struggle with very long sequences. Newer approaches — Rotary Position Embedding (RoPE), ALiBi, and others — are designed to scale to longer contexts and appear in modern LLMs (LLaMA, GPT-4, Claude). Understanding these schemes helps practitioners select models for long-document use cases.

Real-world example: In 'The committee approved the budget after reviewing the concerns', the model needs positional information to understand that 'approved' follows 'committee' as its subject, and 'after reviewing' is a temporal clause, not the main predicate. Without positional embeddings, the Transformer cannot distinguish this from a random shuffling of the same words.

Practical Applications

Retrieval-Augmented Generation (RAG)

RAG is the dominant architecture for grounding language model outputs in specific, up-to-date, or proprietary knowledge. Documents are chunked, embedded, and stored in a vector database. At query time, the user's question is embedded and the most semantically similar document chunks are retrieved. These retrieved chunks are inserted into the language model's context window along with the query, enabling the model to answer based on retrieved facts rather than (only) training data. RAG is the standard solution for the LLM hallucination problem in knowledge-intensive applications.

Semantic Search Over Large Corpora

Traditional keyword search returns documents containing query terms; semantic search returns documents with similar meaning. By embedding all documents and queries into the same vector space, semantic search finds relevant documents even when they use different terminology than the query. Enterprise search, academic literature search, and legal research tools commonly use embedding-based semantic search as either the primary search layer or a re-ranking layer over keyword search results.

Document Clustering and Topic Discovery

Embedding a corpus of documents and then applying clustering algorithms (K-means, HDBSCAN) to the embedding vectors produces thematic groupings without requiring predefined categories. This is used for exploratory data analysis of large text collections — finding the main topics in a corpus of support tickets, news articles, or research papers — without any labelled training data.

Best Practices

- Match the embedding model to the domain and task. General-purpose embedding models work well for general text. For specialised domains (scientific literature, legal documents, code), domain-specific embedding models typically outperform general-purpose ones. Benchmark on a representative sample before committing to a model.
- Store embeddings in a vector database for production retrieval. In-memory similarity search is only viable for small corpora. For production RAG or semantic search, use a purpose-built vector database (Pinecone, Weaviate, Qdrant, pgvector in PostgreSQL) that supports approximate nearest-neighbour search at scale.
- Normalise embeddings before computing cosine similarity. Many vector databases and similarity libraries expect unit-normalised vectors. Verify that the embedding model output is normalised or normalise it yourself before storing, to ensure cosine similarity calculations are correct.
- Re-embed documents when the embedding model is updated. Embeddings from different model versions are not directly comparable. If you upgrade your embedding model, you must re-embed all stored documents to maintain consistency with new query embeddings.

Common Mistakes and Misconceptions

Misconception: Using keyword search where semantic search is needed.

Correct approach: Keyword search fails on synonyms, paraphrases, and terminology mismatches. Semantic embedding-based search is more robust for natural language queries over unstructured text. For most document retrieval applications, a hybrid approach (keyword + semantic) outperforms either alone.

Misconception: Averaging word embeddings as a sentence representation.

Correct approach: Averaging individual word embeddings produces poor sentence representations. Use a dedicated sentence embedding model (sentence-transformers, OpenAI embeddings) that produces a single vector representing the full sentence's meaning in context.

Misconception: Embedding entire long documents as a single vector for RAG.

Correct approach: Long documents lose specificity when compressed into a single embedding vector. For RAG, chunk documents into meaningful segments (paragraphs or semantic units) and embed each chunk separately, enabling precise retrieval of the relevant passage rather than a general document-level match.

Implementation Checklist: Embedding-Based Application Design Checklist

Use this checklist when designing any application that uses embeddings for search, retrieval, or similarity calculations.

- ☐ Select an embedding model appropriate for the domain (general, scientific, legal, multilingual) and benchmark on representative samples.
- ☐ Determine the chunking strategy for long documents — chunk at sentence, paragraph, or section level based on retrieval granularity needs.
- ☐ Choose a vector database appropriate for scale (in-memory for <10K docs, dedicated vector DB for larger corpora).
- ☐ Confirm that stored embeddings and query embeddings use the same model version — mixed versions produce incorrect similarity scores.
- ☐ Implement embedding normalisation if the database requires unit-normalised vectors for cosine similarity.
- ☐ Define the similarity threshold for retrieval — what cosine similarity score constitutes a 'relevant' result for this application?
- ☐ Plan for embedding model updates — document the re-embedding process required when the model is upgraded.
- ☐ Evaluate bias in embedding outputs: test whether semantically equivalent queries produce equivalent results across demographic groups.
- ☐ Monitor retrieval quality in production — log similarity scores and collect user feedback to detect embedding quality degradation.

How to use this: Review this checklist before beginning implementation. Items left unchecked are technical debt — they typically surface as production reliability or quality issues.

Knowledge Check

Test your understanding before moving on. Answers are shown below each question.

1. A developer builds a semantic search system using Word2Vec embeddings. Users report that searches for 'financial institution' return results about river geography. What is the most likely cause?

A. The vector database is malfunctioning.

- B. Word2Vec static embeddings assign the same vector to the word 'bank' regardless of context, so financial and geographical uses of 'bank' have the same representation.
- C. The chunking strategy is incorrect.
- D. Cosine similarity is being computed incorrectly.

Answer: B. Word2Vec static embeddings assign the same vector to the word 'bank' regardless of context, so financial and geographical uses of 'bank' have the same representation.

Why: *Static embeddings cannot distinguish between different senses of a polysemous word. Contextual embeddings from models like BERT or sentence-transformers would produce different vectors for 'bank' in financial versus geographical contexts.*

2. A team upgrades their RAG system to a newer, more accurate embedding model. They notice that retrieval quality drops dramatically after the upgrade. What is the most likely cause?

- A. The new model is less accurate than claimed.
- B. The stored document embeddings were produced by the old model and are incompatible with query embeddings from the new model — the vector spaces are different.
- C. The vector database needs to be restarted after a model upgrade.
- D. Chunking sizes need to be reduced when using newer models.

Answer: B. The stored document embeddings were produced by the old model and are incompatible with query embeddings from the new model — the vector spaces are different.

Why: *Different embedding model versions produce vectors in different spaces. Comparing vectors from different models is meaningless. All stored document embeddings must be re-generated with the new model before queries from the new model will produce correct similarity scores.*

3. Which of the following best explains why Transformer models require positional embeddings?

- A. To reduce the computational cost of self-attention.
- B. To enable the model to process longer sequences.
- C. Because self-attention is permutation-invariant — without positional information, the model cannot distinguish 'the dog bit the man' from 'the man bit the dog'.
- D. To convert token IDs into dense vectors before attention computation.

Answer: C. Because self-attention is permutation-invariant — without positional information, the model cannot distinguish 'the dog bit the man' from 'the man bit the dog'.

Why: *Self-attention computes relationships between all token pairs simultaneously and does not inherently track position. Positional embeddings add sequence order information so the model can encode syntactic and semantic relationships that depend on word order.*

Reflection Questions

1. Embeddings encode semantic relationships that emerge from language patterns — including biases present in the training data. If you were building an embedding-based hiring tool that ranked candidate CVs by similarity to successful employee profiles, what bias risks would you assess before deploying?
2. The analogy 'king - man + woman $\approx$ queen' is often cited as evidence that embeddings capture meaning. What are the limitations of this as evidence? Can you think of cases where the same arithmetic would produce problematic or incorrect results?

3. RAG architectures use embeddings to connect LLMs to external knowledge. What are the failure modes of this approach, and how would you detect them in a production application?
4. Contextual embeddings require a full neural network forward pass for every new piece of text. At what scale does this become a meaningful constraint, and how would you architect around it?

Key Takeaways

- Embeddings are dense numerical vectors that represent text in a high-dimensional space where geometric proximity reflects semantic similarity.
- Well-trained embedding spaces encode semantic relationships as directions — enabling arithmetic on meaning (king - man + woman $\approx$ queen).
- Cosine similarity is the standard metric for comparing embeddings — values closer to 1.0 indicate greater semantic similarity.
- Static embeddings (Word2Vec, GloVe) assign one vector per word regardless of context; contextual embeddings (BERT, GPT) capture polysemy by encoding word meaning in context.
- Positional embeddings inject sequence order information into Transformer models, which otherwise cannot distinguish word order.
- Embedding APIs enable semantic search, RAG, document clustering, and recommendation systems by converting any text into comparable vectors.
- Production embedding applications require a vector database for scalable nearest-neighbour search, and all stored embeddings must be re-generated when the embedding model is updated.

Recommended Next Actions

5. Explore an embedding visualisation: use the TensorFlow Embedding Projector (projector.tensorflow.org) to see how words cluster in 2D or 3D projections of a pre-trained embedding space.
6. Try computing cosine similarity between sentence pairs using sentence-transformers (`pip install sentence-transformers`) and test whether the similarity scores match your intuition about which sentences are related.
7. Identify one use case in your work or domain where semantic search would be more useful than keyword search. Describe what you would embed, how you would store the vectors, and what queries you would run.
8. Add to your glossary: Embedding, Vector Space, Cosine Similarity, Positional Embedding, Static vs Contextual Embeddings, RAG, Vector Database.